

ЗВІТ

до лабораторної роботи № 1

**Розробка елементів експертної системи продукційного типу,
яка реалізує пряме виведення**

Предметна область: діагностика типових несправностей персонального
комп'ютера

CONTENTS

1. Мета роботи	3
2. Постановка завдання.....	4
3. Короткі теоретичні відомості.....	5
4. Опис предметної області	6
5. База знань продукційного типу	7
6. Структури даних і алгоритм прямого виведення	8
7. Програмна реалізація мовою Python	9
8. Перевірка роботи експертної системи	12
9. Аналіз результатів.....	13
10. Висновки	13

1. МЕТА РОБОТИ

Вивчити склад і принципи роботи експертної системи продукційного типу та реалізувати механізм прямого логічного виведення. Побудувати базу знань у формі продукційних правил, організувати робочу пам'ять, чергу змінних і протокол спрацювання правил, а також перевірити роботу системи на тестових прикладах.

2. ПОСТАНОВКА ЗАВДАННЯ

Необхідно розробити експертну систему продукційного типу для обраної предметної області. База знань повинна містити від 6 до 10 правил, що відображають причинно-наслідкові зв'язки. Система має реалізовувати пряме виведення, підтримувати введення вихідних фактів, формувати остаточний результат і за вимогою показувати ланцюг логічних висновків.

3. КОРОТКІ ТЕОРЕТИЧНІ ВІДОМОСТІ

Експертна система складається з бази знань, робочої пам'яті та механізму логічного виведення. База знань містить продукційні правила виду «ЯКЩО умова, ТО висновок». Робоча пам'ять зберігає факти, відомі системі в поточний момент, а механізм виведення визначає, які правила можуть бути активовані.

За прямого виведення процес починається з відомих фактів. Після спрацювання правила новий факт додається до робочої пам'яті, після чого система перевіряє наступні правила. Такий підхід називають виведенням, керованим даними. Для пошуку завширшки нові факти опрацьовуються в порядку їх надходження за принципом FIFO. На рис. 1.1 узагальнена структура експертної системи.

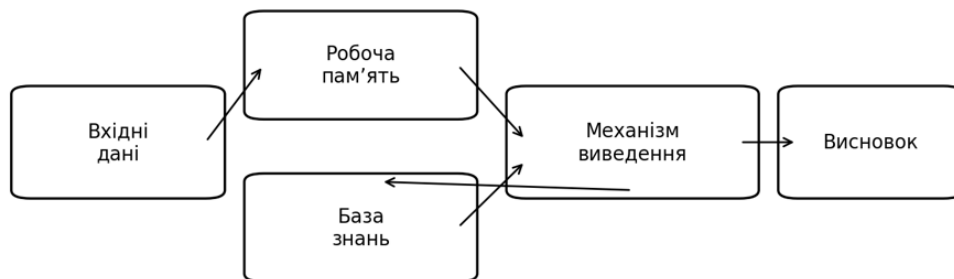


Рисунок 3.1 – Узагальнена структура експертної системи

4. ОПИС ПРЕДМЕТНОЇ ОБЛАСТІ

Предметною областю обрано первинну діагностику несправностей персонального комп'ютера. Користувач повідомляє спостережувані ознаки: чи вмикається живлення, чи обертаються вентилятори, чи є зображення, чи завантажуються операційна система, чи визначається накопичувач і чи доступна мережа. На основі цих фактів експертна система формує один або декілька діагностичних висновків.

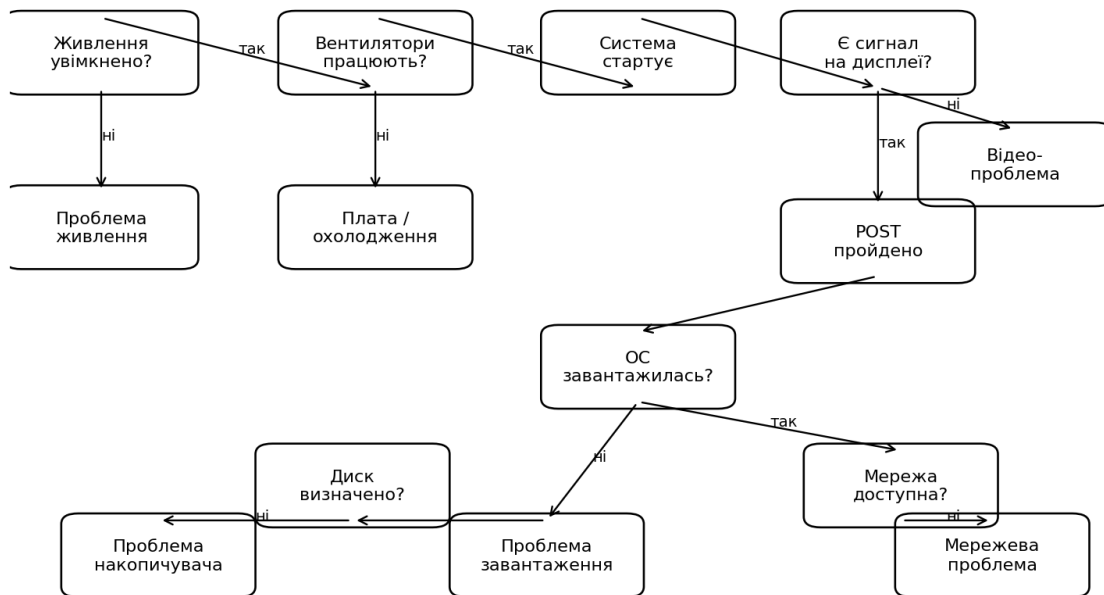


Рисунок 2 – Фрагмент дерева логічних висновків для діагностики ПК

5. БАЗА ЗНАНЬ ПРОДУКЦІЙНОГО ТИПУ

Для експертної системи сформовано дев'ять продукційних правил:

№	Умовна частина	Висновок
R1	живлення = ні	проблема_живлення = так.
R2	живлення = так І вентилятори = ні	проблема_плати_або_охолодження = так.
R3	живлення = так І вентилятори = так	система_стартує = так.
R4	система_стартує = так І сигнал_дисплея = ні	проблема_відеосистеми = так.
R5	система_стартує = так І сигнал_дисплея = так	post_пройдено = так.
R6	post_пройдено = так І ос_завантажилась = ні	проблема_завантаження = так.
R7	проблема_завантаження = так І диск_визначено = ні	проблема_накопичувача = так.
R8	post_пройдено = так І ос_завантажилась = так	ос_працює = так.
R9	ос_працює = так І мережа_доступна = ні	мережева_проблема = так.

Таблиця 1 – База знань експертної системи

6. СТРУКТУРИ ДАНИХ І АЛГОРИТМ ПРЯМОГО ВИВЕДЕННЯ

У програмі база знань подається списком об'єктів Rule. Кожне правило містить набір умов, висновок та ознаку fired, яка показує, чи було правило вже активовано. Робоча пам'ять реалізована словником facts, де ключем є ім'я змінної, а значенням – її поточне значення. Черга queue зберігає нові факти в порядку FIFO, а список explanation накопичує текстовий протокол спрацювання правил.

1. Записати вихідні факти користувача до робочої пам'яті та черги.
2. Взяти з початку черги черговий факт.
3. Переглянути правила, в умовах яких зустрічається ця змінна.
4. Для кожного правила перевірити, чи відомі всі його умови та чи всі вони істинні.
5. Якщо правило істинне і ще не спрацювало, додати його висновок до робочої пам'яті й у кінець черги.
6. Записати спрацювання правила до ланцюга пояснень.
7. Повторювати кроки 2–6, поки черга не стане порожньою.
8. Вивести діагностичні висновки та ланцюг логічного виведення.

7. ПРОГРАМНА РЕАЛІЗАЦІЯ МОВОЮ PYTHON

```
from dataclasses import dataclass
from collections import deque

@dataclass
class Rule:
    name: str
    conditions: dict
    conclusion: tuple
    fired: bool = False

RULES = [
    Rule(
        "R1",
        {"power_on": False},
        ("power_problem", True)
    ),

    Rule(
        "R2",
        {"power_on": True, "fans_spin": False},
        ("board_or_cooling_problem", True)
    ),

    Rule(
        "R3",
        {"power_on": True, "fans_spin": True},
        ("system_started", True)
    ),

    Rule(
        "R4",
        {"system_started": True, "display_signal": False},
        ("video_problem", True)
    ),

    Rule(
        "R5",
        {"system_started": True, "display_signal": True},
        ("post_passed", True)
    ),

    Rule(
        "R6",
        {"post_passed": True, "os_booted": False},
        ("boot_problem", True)
    ),

    Rule(
        "R7",
        {"boot_problem": True, "disk_detected": False},
        ("storage_problem", True)
    ),
]
```

```

Rule(
    "R8",
    {"post_passed": True, "os_booted": True},
    ("os_working", True)
),

Rule(
    "R9",
    {"os_working": True, "network_available": False},
    ("network_problem", True)
),
]

def infer(initial_facts):
    facts = dict(initial_facts)

    # Черга фактів, які потрібно обробити
    queue = deque(initial_facts.keys())

    explanation = []

    # Скидання стану правил
    for rule in RULES:
        rule.fired = False

    while queue:
        variable = queue.popleft()

        for rule in RULES:

            # Якщо правило вже спрацювало
            # або поточна змінна не бере участі в правилі
            if rule.fired or variable not in rule.conditions:
                continue

            # Перевіряємо, чи всі необхідні факти вже відомі
            all_known = all(
                k in facts
                for k in rule.conditions
            )

            # Головне виправлення:
            # перевіряємо значення тільки якщо всі факти існують
            if all_known and all(
                facts[k] == v
                for k, v in rule.conditions.items()
            ):

                result_var, result_value = rule.conclusion

                rule.fired = True

                if result_var not in facts:
                    facts[result_var] = result_value
                    queue.append(result_var)

                explanation.append(

```

```

        f"{rule.name}: {rule.conditions} -> "
        f"{result_var}={result_value}"
    )

    return facts, explanation

def ask_yes_no(question):
    while True:
        value = input(
            question + " [так/ні]: "
        ).strip().lower()

        if value in ("так", "т", "yes", "y"):
            return True

        if value in ("ні", "н", "no", "n"):
            return False

        print("Введіть 'так' або 'ні'.")

def main():
    initial = {
        "power_on": ask_yes_no(
            "Чи вмикається живлення?"
        ),
        "fans_spin": ask_yes_no(
            "Чи працюють вентилятори?"
        ),
        "display_signal": ask_yes_no(
            "Чи є сигнал на дисплеї?"
        ),
        "os_booted": ask_yes_no(
            "Чи завантажилась ОС?"
        ),
        "disk_detected": ask_yes_no(
            "Чи визначено накопичувач?"
        ),
        "network_available": ask_yes_no(
            "Чи доступна мережа?"
        ),
    }

    facts, explanation = infer(initial)

    diagnoses = {
        "power_problem":
            "Проблема живлення",

        "board_or_cooling_problem":
            "Проблема системної плати або охолодження",
    }

```

```

"video_problem":
    "Проблема відеосистеми",

"boot_problem":
    "Проблема завантаження ОС",

"storage_problem":
    "Проблема накопичувача",

"network_problem":
    "Проблема мережевого підключення",
}

print("\nРезультат експертизи:")

found = False

for key, label in diagnoses.items():

    if facts.get(key) is True:
        print("-", label)
        found = True

if not found:
    print(
        "Критичних несправностей за заданими "
        "правилами не виявлено."
    )

print("\nЛанцюг логічних висновків:")

for step in explanation:
    print(step)

if __name__ == "__main__":
    main()

```

8. ПЕРЕВІРКА РОБОТИ ЕКСПЕРТНОЇ СИСТЕМИ

Для перевірки реалізації використано три тестові набори вихідних фактів. У кожному випадку система повинна отримати діагноз за допомогою послідовного спрацювання продукційних правил.

Тест	Вихідні факти	Ланцюг правил	Отриманий висновок
1	Живлення = ні	R1	Проблема живлення
2	Живлення = так; вентилятори = так; сигнал дисплея = так; ОС = ні; диск = ні	$R3 \rightarrow R5 \rightarrow R6 \rightarrow R7$	Проблема завантаження ОС; проблема накопичувача

3	Живлення = так; вентилятори = так; сигнал дисплея = так; ОС = так; мережа = ні	$R3 \rightarrow R5 \rightarrow R8 \rightarrow R9$	Проблема мережевого підключення
---	---	---	------------------------------------

Таблиця 2 – Результати тестування

Приклад роботи для тесту 2:

Чи вмикається живлення? [так/ні]: так
 Чи працюють вентилятори? [так/ні]: так
 Чи є сигнал на дисплеї? [так/ні]: так
 Чи завантажилась ОС? [так/ні]: ні
 Чи визначено накопичувач? [так/ні]: ні
 Чи доступна мережа? [так/ні]: так

Результат експертизи:

- Проблема завантаження ОС
- Проблема накопичувача

Ланцюг логічних висновків:

R3: power_on=True, fans_spin=True -> system_started=True
 R5: system_started=True, display_signal=True -> post_passed=True
 R6: post_passed=True, os_booted=False -> boot_problem=True
 R7: boot_problem=True, disk_detected=False -> storage_problem=True

9. АНАЛІЗ РЕЗУЛЬТАТІВ

У всіх тестах отримані результати відповідають сформованій базі знань. У тесті 2 висновок утворюється не одним правилом, а послідовністю R3, R5, R6 і R7, що підтверджує роботу прямого виведення. Нові факти system_started, post_passed і boot_problem послідовно додаються до робочої пам'яті та черги, після чого стають підставою для активації наступних правил.

Реалізація відокремлює знання від механізму виведення: для розширення предметної області достатньо додати нові об'єкти Rule без зміни функції infer(). Це забезпечує модульність продукційної моделі та спрощує подальше доповнення системи.

10. ВИСНОВКИ

У лабораторній роботі розроблено експертну систему продукційного типу для первинної діагностики несправностей персонального комп'ютера. Сформовано базу знань з дев'яти правил, реалізовано робочу пам'ять, чергу FIFO та механізм прямого логічного виведення мовою Python. Система приймає вихідні факти,

формує діагностичний висновок і відображає ланцюг правил, які привели до результату. Тестування показало коректне послідовне спрацювання правил і можливість одержання багатокрокових висновків.